

Interim Report for Evaluation (Phase III)

Drivo: Driving School Management System

Submitted in partial fulfillment of the requirement for the award of

MASTER OF SCIENCE IN COMPUTER SCIENCE

(UNIVERSITY OF CALICUT)

Submitted by

**MOHAMMED ASIF.A
(Reg. No:FKAYMCS007)**

**Under the Guidance of
BODHY KRISHNA.S**

2024-2026



**Department of Computer Science
FAROOK COLLEGE
(Autonomous)
FAROOK COLLEGE (P.O), CALICUT**

**Farook College
(Autonomous)**

Certificate from the Guide

1. Name of the Student	MOHAMMED ASIF.A
2. Roll Number	85
3. Title of the Project	DRIVO:DRIVING SCHOOL MANAGEMENT SYSTEM
4. Project Category	MOBILE APPLICATION
5. Guide	BODHY KRISHNA.S

Draft copy of this report was submitted to me on (specify the date of submission to the guide) and my reviews and comments are included in the final report.

Signature of the Guide

Date:

TABLE OF CONTENTS

Sl No	Content	Page No
1	Introduction	
2	Tools/Scripts used for implementation	
3	Module Hierarchy	
4	Details of the user	
5	Coding	
6	Screenshots	
7	Bibliography	

1. Introduction

Drivo: Driving School Management System is a web-based application designed to simplify and automate the operations of driving schools. The system provides an efficient platform for managing student registrations, scheduling driving lessons, tracking learner progress, and handling instructor assignments. Traditional driving school management often involves manual processes, which can lead to errors, delays, and inefficiencies. Drivo addresses these challenges by offering a centralized digital solution that ensures accurate data management, easy accessibility, and improved communication between learners, instructors, and administrators. The application is built using modern web technologies, providing a user-friendly interface and secure data handling. It enables administrators to manage all activities effectively while allowing students to track their learning progress and schedule sessions conveniently. Overall, Drivo aims to enhance operational efficiency, reduce manual workload, and provide a seamless experience for both driving schools and learners.

2. Tools/Script used for implementation

The project “Drivo: Driving School Management System” is developed as a cross-platform mobile application using modern frameworks and cloud-based technologies. The system follows a serverless architecture to ensure scalability, real-time data handling, and efficient performance. The following tools and technologies are used for implementation:

2.1 Flutter

Flutter is an open-source UI framework used for developing cross-platform mobile applications from a single codebase. It uses the Dart programming language and provides a rich set of widgets for building responsive and visually appealing user interfaces.

In this project, Flutter is used to develop the complete mobile application for both students and administrators. It enables features such as user registration, course enrollment, booking driving sessions, test scheduling, and performance tracking. Flutter ensures fast development, high performance, and a consistent user experience across platforms like Android, iOS, and Web.

2.2 Android

Android is one of the primary platforms on which the Drivo application is deployed. It provides the runtime environment required for executing the mobile application on smartphones.

In this project, Android is used as the target platform for running the Flutter-based application. It allows users to access the system easily through mobile devices, ensuring convenience, accessibility, and real-time interaction with the system

2.3 Firebase (Cloud Firestore & Backend Services)

Firebase is a Backend-as-a-Service (BaaS) platform used to provide serverless backend support for the application. It offers various services such as real-time database, authentication, and cloud storage.

In this project, Firebase is used for:

- **Cloud Firestore:** A NoSQL real-time database used to store user data, booking details, test schedules, and application status
- **Authentication:** Secure login and registration of users and administrators
- **Real-time Data Sync:** Ensures instant updates and seamless data flow across the application

Firebase eliminates the need for a traditional backend server and ensures scalability, security, and high performance.

2.4 State Management (Provider)

Provider is a state management technique used in Flutter to manage application data and separate business logic from the user interface.

In this project, Provider is used to handle application states efficiently through controllers such as user and admin controllers. It helps in maintaining clean code structure, improving performance, and ensuring better maintainability of the application.

2.5 Cloudinary (Media Storage API)

Cloudinary is a cloud-based media management service used for storing and delivering images and files efficiently.

In this project, Cloudinary is used to upload and manage user-related media such as profile images and documents. It reduces the load on Firebase storage and ensures fast and optimized media delivery

3. Module Hierarchy

The “Drivo: Driving School Management System” is designed using a modular approach to ensure efficient functionality, scalability, and ease of maintenance. The system is mainly divided into two major modules: User (Student) Module and Admin Module, each handling specific tasks within the application.

3.1 User (Student) Module

This module is designed for students who enroll in driving courses. It provides all functionalities required for managing their learning process.

Key functionalities include:

- **User Registration & Login:** Allows users to create an account and securely log in using authentication services
- **Profile Management:** Enables users to update personal details and upload necessary documents
- **Course Enrollment:** Users can view available driving courses and enroll in suitable programs
- **Booking Driving Sessions:** Allows users to schedule driving classes based on availability
- **Test Scheduling:** Enables users to book driving tests and view test dates
- **Retest Management:** Provides options to reschedule or apply for retests if needed
- **Payment Management:** Facilitates secure payment for courses and services
- **Mock Test:** Provides practice tests for learners to improve their knowledge of driving rules and prepare for the actual driving test
- **Invoice Downloading:** Allows users to download payment receipts/invoices for completed transactions

- **Announcements:** View important updates and notifications from the admin (e.g., schedule changes, test dates, holidays)
- **Feedback:** Users can provide feedback about training, instructors, and overall experience

3.2 Admin Module

This module is designed for administrators to manage and control the overall system efficiently.

Key functionalities include:

- **Admin Login:** Secure authentication for administrative access
- **User Management:** View, add, update, or remove student details
- **Course Management:** Create, update, and manage available driving courses
- **Schedule Management:** Manage driving sessions, instructor allocation, and timing
- **Test Management:** Schedule driving tests and manage test records
- **Payment Monitoring:** Track user payments and transaction details
- **Application Tracking:** Monitor user registrations, approvals, and status updates
- **Attendance Marking:** Record and manage student attendance for sessions
- **Announcement Management:** Create and publish announcements for users
- **Invoice Management:** Generate and manage invoices
- **Support Management:** View and respond to user queries and issues
- **Reports & Analytics:** Generate reports related to users, performance, and system activities

3.3 Authentication & Data Management Module

This module handles backend services using Firebase.

Key functionalities include:

- **User Authentication:** Secure login and registration using Firebase Authentication
- **Database Management:** Store and manage data using Cloud Firestore
- **Real-Time Updates:** Synchronize data instantly across users and admin

- **Data Security:** Ensure safe storage and controlled access to data

4. Details of the Users of the System

The “**Drivo: Driving School Management System**” is designed with role-based access to ensure secure and efficient operation. The system consists of four main types of users: **Student, User, Instructor, and Administrator**. Each user has specific roles and responsibilities within the system.

4.1 Student

Students are the primary users who enroll in driving courses and participate in training sessions.

Responsibilities and functionalities include:

- Register and log in to the application
- Enroll in driving courses
- Book driving sessions and schedule tests
- Attend mock tests for practice
- View announcements and updates
- Track learning progress and performance
- Download certificates after course completion

4.2 User

Users are general app users who access various services provided by the system.

Responsibilities and functionalities include:

- Register and log in to the system
- View available courses and details
- Apply for Driving Test through the application
- RC Renewal: Apply for renewal of vehicle registration certificate
- Access general driving-related services and information

- Contact support and raise queries
- View announcements and updates

4.3 Instructor

Instructors are responsible for providing driving training and monitoring student performance.

Responsibilities and functionalities include:

- View assigned students and schedules
- Conduct driving sessions
- Mark attendance for students
- Provide feedback on student performance
- Assist students during training
- Update session details and progress

4.4 Administrator (Admin)

Administrators manage and control the overall system operations.

Responsibilities and functionalities include:

- Secure login to the system
- Manage users, students, and instructors
- Create and manage driving courses
- Schedule sessions and allocate instructors
- Monitor attendance and performance
- Manage tests and retests
- Handle payments and generate invoices
- Publish announcements
- Handle support requests and feedback
- Generate reports and monitor system performance

5. Coding

User:

```
import 'package:driving_school/const.dart';
import 'package:driving_school/controller/user_controller.dart';
import 'package:driving_school/services/certificate_service.dart'; // Added correctly
import 'package:driving_school/controller/admin_controller.dart';
import 'package:driving_school/views/user/user_settings.dart';
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';
import 'package:provider/provider.dart';
import 'package:intl/intl.dart';
import 'package:easy_localization/easy_localization.dart'; // Added
import 'package:driving_school/views/user/widgets/user_header_widget.dart';
import 'package:driving_school/views/user/widgets/dashboard_grid_widget.dart';

class UserHome extends StatefulWidget {
  final String uid;
  const UserHome({required this.uid, super.key});

  @override
  State<UserHome> createState() => _UserHomeState();
}

class _UserHomeState extends State<UserHome> {

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      Provider.of<AdminController>(context, listen: false).fetchAnnouncements();
    });
  }

  @override
```

```

Widget build(BuildContext context) {
  final height = MediaQuery.of(context).size.height;
  final width = MediaQuery.of(context).size.width;
  final userHomeController = Provider.of<UserController>(context);
  final adminController = Provider.of<AdminController>(context);

  print('HOME PAGE');
  return Scaffold(
    body: Stack(
      children: [
        // Background Elements
        Positioned(
          top: 0,
          right: 0,
          child: Image.asset('assets/Ellipse 2.png'),
        ),
        Positioned(
          left: 0,
          right: 0,
          top: 200,
          child: Opacity(
            opacity: 0.5,
            child: Image.asset(
              'assets/Ellipse 36.png',
              fit: BoxFit.cover,
            ),
          ),
        ),
      ],
    ),
    // Main Content
    FutureBuilder(
      future: userHomeController.fetchUserData(widget.uid),
      builder: (context, snapshot) {
        return snapshot.connectionState == ConnectionState.waiting
          ? const Center(
              child: CircularProgressIndicator(

```

```

        strokeWidth: 2,
        color: defaultBlue,
      ),
    )
  : SafeArea(
    child: SingleChildScrollView(
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 20),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const SizedBox(
              height: 20,
            ),
            // Header Section
            SizedBox(
              width: width,
              child: UserHeaderWidget(userController: userHomeController),
            ),

            // ANNOUNCEMENT SECTION
            if (adminController.announcementList.isNotEmpty)
              Padding(
                padding: const EdgeInsets.only(top: 20),
                child: Container(
                  width: width,
                  padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 15),
                  decoration: BoxDecoration(
                    color: const Color(0xFFFF5F9FF), // Very soft blue background
                    borderRadius: BorderRadius.circular(20),
                    border: Border.all(color: Colors.blue.withOpacity(0.1)),
                  ),
                ),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Row(

```

```

children: [
  Container(
    padding: const EdgeInsets.all(6),
    decoration: BoxDecoration(
      color: Colors.blue.withOpacity(0.1),
      shape: BoxShape.circle,
    ),
    child: const Icon(Icons.notifications_none_rounded, color:
Colors.blue, size: 18),
  ),
  const SizedBox(width: 10),
  Expanded(
    child: Text(
      adminController.announcementList.first.title,
      style: GoogleFonts.epilogue(
        fontWeight: FontWeight.w700,
        fontSize: 13,
        color: Colors.blue.shade900,
        letterSpacing: 0.5,
      ),
    ),
  ),
  if (adminController.announcementList.first.date.isNotEmpty)
    Container(
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(10),
      ),
      child: Text(
        adminController.announcementList.first.date,
        style: GoogleFonts.epilogue(
          fontSize: 10,
          fontWeight: FontWeight.w600,
          color: Colors.grey.shade500,
        ),
      ),
    ),

```

```

        ),
    ),
],
),
const SizedBox(height: 8),
Padding(
  padding: const EdgeInsets.only(left: 38), // Align with text start
  child: Text(
    adminController.announcementList.first.description,
    style: GoogleFonts.epilogue(
      fontSize: 13,
      color: Colors.blueGrey.shade700,
      height: 1.4,
    ),
  ),
),
),
],
),
),
),
),

const SizedBox(height: 15),

// Course Card (Smaller)
Container(
  width: width,
  height: height / 7.5, // Smaller height
  decoration: BoxDecoration(
    gradient: const LinearGradient(
      colors: [defaultBlue, Color(0xFF2979FF)],
      begin: Alignment.topLeft,
      end: Alignment.bottomRight,
    ),
  ),
  borderRadius: BorderRadius.circular(22),
  boxShadow: [
    BoxShadow(

```

```
        color: defaultBlue.withOpacity(0.3),
        blurRadius: 12,
        offset: const Offset(0, 6),
      ),
    ],
  ),
  child: Stack(
    children: [
      Positioned(
        right: -15,
        top: -15,
        child: Container(
          width: 80,
          height: 80,
          decoration: BoxDecoration(
            color: Colors.white.withOpacity(0.1),
            shape: BoxShape.circle,
          ),
        ),
      ),
      Positioned(
        left: -25,
        bottom: -25,
        child: Container(
          width: 120,
          height: 120,
          decoration: BoxDecoration(
            color: Colors.white.withOpacity(0.05),
            shape: BoxShape.circle,
          ),
        ),
      ),
    ],
  ),
  child: Padding(
    padding: const EdgeInsets.all(20.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
```

```

mainAxisAlignment: MainAxisAlignment.spaceBetween,
children: [
  Container(
    padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
    decoration: BoxDecoration(
      color: Colors.white.withOpacity(0.2),
      borderRadius: BorderRadius.circular(15),
    ),
    child: Text(
      "select_course".tr(), // Using translation
      style: GoogleFonts.epilogue(
        color: Colors.white,
        fontSize: 9,
        fontWeight: FontWeight.bold,
        letterSpacing: 0.5,
      ),
    ),
  ),
  Text(
    userHomeController.userModel.selectedCourse ?? 'No Class Selected',
    style: GoogleFonts.epilogue(
      color: Colors.white,
      fontSize: 20, // Slightly smaller font
      fontWeight: FontWeight.bold,
    ),
  ),
],
),
const SizedBox(height: 15),

// Service Categories
Text(

```



```

        "Service Categories:", // Ideally add a JSON key for this later if needed
        style: GoogleFonts.epilogue(
          fontSize: 18,
          fontWeight: FontWeight.w500,
        ),
      ),
      const SizedBox(height: 15),

      // Service Grid
      // Service Grid
      DashboardGridWidget(userController: userHomeController),
      const SizedBox(height: 20,)
    ],
  ),
),
),
);
},
),
],
)
);
}
}

```

Main.dart:

```

import 'package:driving_school/controller/admin_controller.dart';
import 'package:driving_school/controller/payment_gateway.dart';
import 'package:driving_school/controller/user_controller.dart';
import 'package:driving_school/controller/mock_test_controller.dart';
import 'package:driving_school/firebase_options.dart';
import 'package:driving_school/views/splash_screen.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:easy_localization/easy_localization.dart';

```

```
import 'package:driving_school/services/notification_service.dart';
```

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await EasyLocalization.ensureInitialized();  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
  
  final notificationService = NotificationService();  
  await notificationService.initNotification();  
  await notificationService.requestPermissions();
```

```
  runApp(  
    EasyLocalization(  
      supportedLocales: const [Locale('en', 'US')],  
      path: 'assets/translations',  
      fallbackLocale: const Locale('en', 'US'),  
      child: const MyApp(),  
    ),  
  );  
}
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MultiProvider(  
      providers: [  
        ChangeNotifierProvider<UserController>(  
          create: (context) => UserController(),  
        ),  
        ChangeNotifierProvider<PaymentGateway>(  
          create: (context) => PaymentGateway(),
```

```

    ),
    ChangeNotifierProvider<AdminController>(
      create: (context) => AdminController(),
    ),
    ChangeNotifierProvider<MockTestController>(
      create: (context) => MockTestController(),
    ),
  ],
  child: MaterialApp(
    localizationsDelegates: context.localizationDelegates,
    supportedLocales: context.supportedLocales,
    locale: context.locale,
    debugShowCheckedModeBanner: false,

    title: 'Driving School',
    theme: ThemeData(
      colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
      useMaterial3: true,
    ),
    home: const SplashScreen(),
  ),
);
}
}

```

Attendance Marking:

```

import 'package:driving_school/controller/admin_controller.dart';
import 'package:driving_school/controller/user_controller.dart';
import 'package:driving_school/views/admin/add_course.dart';
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';
import 'package:icons_plus/icons_plus.dart';
import 'package:provider/provider.dart';
import 'package:table_calendar/table_calendar.dart';

```

```

class AttendanceMarking extends StatelessWidget {
  final String userID;

```

```

final String userName;
final int userNumber;
const AttendanceMarking({
  required this.userID,
  required this.userName,
  required this.userNumber,
  super.key,
});

```

@override

```

Widget build(BuildContext context) {
  final height = MediaQuery.of(context).size.height;
  final width = MediaQuery.of(context).size.width;
  // final attendanceController = Provider.of<UserController>(context);
  return Scaffold(
    body: Stack(
      children: [
        //////////////////////////////////////
        Positioned(
          top: 0, right: 0, child: Image.asset('assets/Ellipse 2.png')),
        // Column(
        //   mainAxisAlignment: MainAxisAlignment.center,
        //   crossAxisAlignment: CrossAxisAlignment.start,
        //   children: [Image.asset('assets/Ellipse 36.png')]),
        //////////////////////////////////////
        Padding(
          padding: const EdgeInsets.symmetric(
            horizontal: 20,
          ),
          child: Column(
            children: [
              SizedBox(
                width: width,
                height: height / 6,
                child: Row(
                  // crossAxisAlignment: CrossAxisAlignment.end,

```

```

children: [
  IconButton(
    onPressed: () {
      Navigator.of(context).pop();
    },
    icon: const Icon(EvaIcons.arrow_ios_back_outline),
  ),
  const SizedBox(
    width: 20,
  ),
  Text(
    'Attendance Marking',
    style: GoogleFonts.epilogue(
      fontSize: 20,
      fontWeight: FontWeight.w500,
    ),
  ),
],
),
),
Container(
  alignment: Alignment.topLeft,
  width: width,
  height: 50,
  child: Text(
    'Select a Slot',
    style: GoogleFonts.epilogue(
      fontSize: 20, fontWeight: FontWeight.w500),
  ),
),
Consumer<AdminController>(
  builder: (context, attendanceController, _) {
return FutureBuilder(
  future: attendanceController.fetchAttendance(userID),
  builder: (context, snapshot) {
    return TableCalendar(

```

```

eventLoader: (day) {
    return attendanceController.userAttendance.entries
        .map((entry) => entry.value[day] ?? [])
        .expand((event) => event)
        .toList();
},
firstDay: DateTime(2000),
lastDay: DateTime(2100),
focusedDay: attendanceController.today,
onDaySelected: (selectedDay, focusedDay) {
    attendanceController.onDaySelected(
        selectedDay, focusedDay, userID);

    // print(object)
},
availableGestures: AvailableGestures.horizontalSwipe,
selectedDayPredicate: (day) =>
    isSameDay(day, attendanceController.today),
);
});
)),
Expanded(child: Consumer<AdminController>(
    builder: (context, attendanceListController, _) {
return attendanceListController.userAttendance.isEmpty
    ? const Center(
        child: Text('No Attendance'),
    )
: ListView.separated(
    itemBuilder: (context, index) {
        String userAttID = attendanceListController
            .userAttendance.keys
            .elementAt(index);

        List<DateTime> userAttDates =
            attendanceListController
                .userAttendance[userID]!.keys
                .toList();

```

```

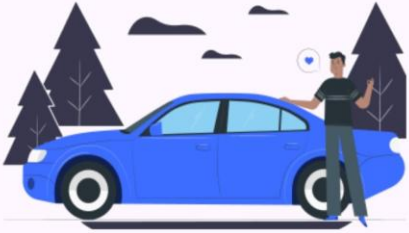
return Padding(
  padding:
    const EdgeInsets.symmetric(horizontal: 10),
  child: Card(
    child: ListTile(
      leading: Text('User : $userName'),
      title: Text('Date : $userAttDates'),
    ),
  ),
);
},
separatorBuilder: (context, index) => const SizedBox(
  height: 10,
),
itemCount:
  attendanceListController.userAttendance.length);
)))
],
),
),
],
),
);
}
}

```

6. Screenshots

6.1 .Admin

ADMIN Login



Enter ID

Enter password

Verify

Welcome back,
Admin,

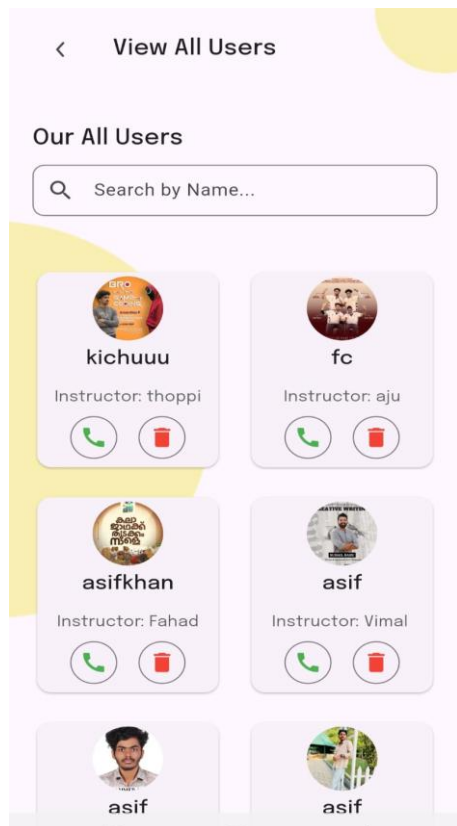
Logout

Overview

27	16	19	₹524
Students	Instructors	Courses	Revenue

Service Categories:

 Users	 Manage Course	 Manage Invoice
 Manage Instructor	 Manage RC Renewal	 FAQ & Feedback
 Manage Slot	 View Bookings	 Test Management



Manage RC Renewals

Vehicle Class: **BIKE**

Engine No: **gbnmm**

Chassis No: **hhmooo**

Amount: **₹ 500.0**

Applied On: **31-12-2025**

View ID Proof

View Pollution Cert.

fabeeb

Pending

Phone: **6325698668**

RC Number: **hkkih**

Expiry Date: **30-12-2025**

Vehicle Class: **LMV**

Engine No: **ghnj**

Chassis No: **ghij**

Amount: **₹ 500.0**

Applied On: **30-12-2025**

View ID Proof

View Pollution Cert.

Manage Announcements

Post New Notice

Title

Message

Post Notice

1234567890

qwertyuiop

asdfghjkl

↵

zxcvbnm

⌫

!#1

,

English(India)

.

Done

Study Material

Q&A

Traffic Signs

Q1. What is the minimum age to obtain a learner's license for a motorcycle without gear?

▼

Q2. What should you do when you see a pedestrian waiting at a zebra crossing?

▼

Q3. When approaching a curve, you should:

▼

Q4. What indicates a 'Slippery Road' sign?

▼

Q5. The validity of a Learner's License is:

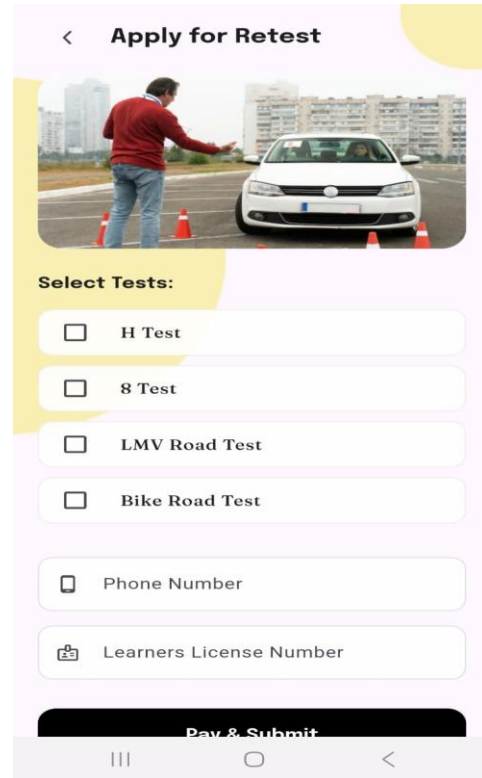
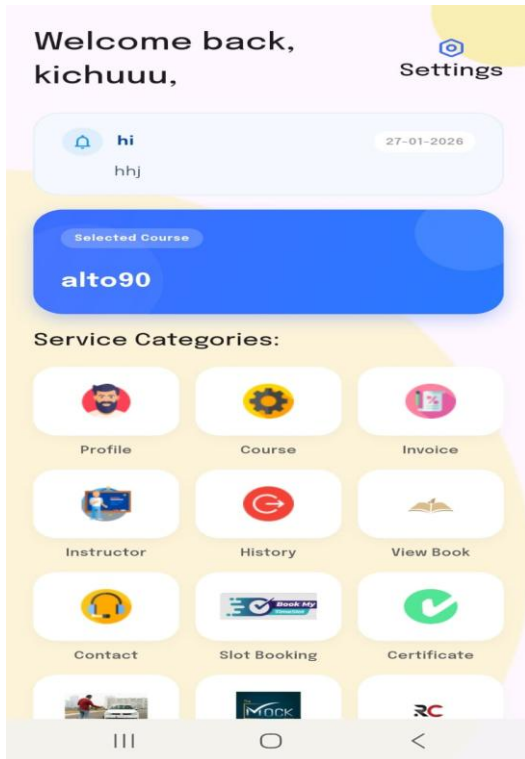
▼

Q6. Driving without insurance is:

▼

Q7. What does a continuous yellow line on the road mean?

▼



Bibliography

- [1] Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2016.
- [2] Roger S. Pressman and Bruce R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw-Hill Education, 2015.
- [3] Herbert Schildt, *Java: The Complete Reference*, 11th Edition, McGraw-Hill Education, 2019.
- [4] Android Developers, "Android Developer Documentation," Available: <https://developer.android.com>
- [5] Firebase Documentation, "Firebase for Mobile Applications," Available: <https://firebase.google.com/docs>
- [6] W3Schools, "Web and Mobile Development Tutorials," Available: <https://www.w3schools.com>
- [7] TutorialsPoint, "Mobile Application Development and Database Concepts," Available: <https://www.tutorialspoint.com>
- [8] Google Developers, "Mobile Application Development Guidelines,